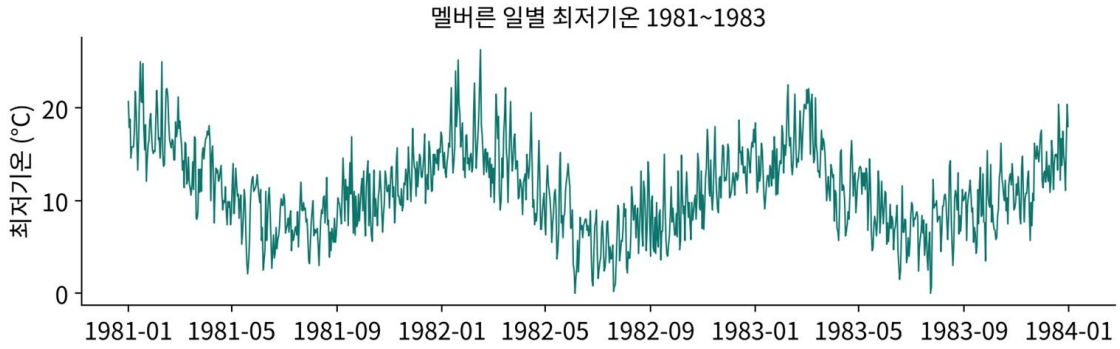
RNN 복습:
시계열 데이터와 순환 신경망

1

1일차: 시계열 데이터와 RNN

순서가 있는 데이터를 어떻게 읽을까

시계열 데이터: 순서가 곧 정보



데이터	한 시점의 값
기온, 주가, 전력량	숫자 1개 (또는 여러 개)
음성	그 순간의 소리 크기
문장	단어 하나

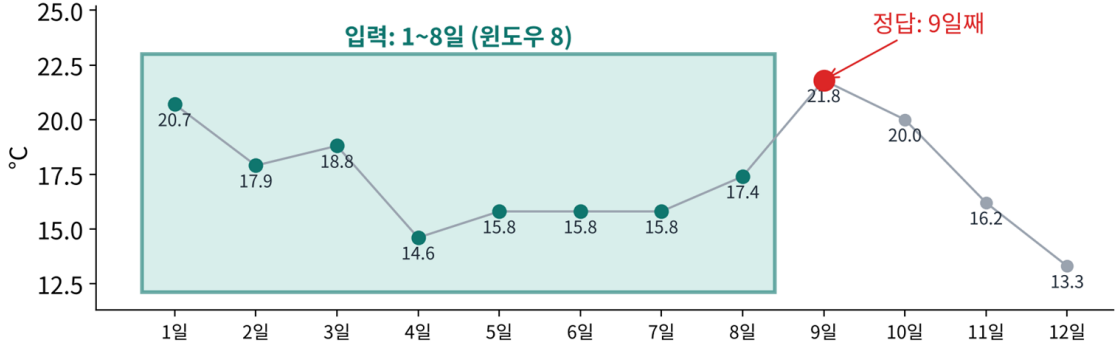
핵심

순서를 섞으면 의미가 사라짐.
'개가 사람을 물었다' ≠ '사람이 개를 물었다'
어제·오늘 기온을 알면 내일 기온을 짐작할 수 있음.

이미지와 차이

이미지: 위치(가로·세로)가 중요 → CNN
시계열: 순서(시간)가 중요 → RNN

예측 문제 만들기: 윈도우(window)



지도학습으로 바뀜: 입력 X = 과거 W개, 정답 y = 다음 값. 정답을 따로 만들 필요 없이 데이터 자체에서 나옴.

문제 번호	입력 X (8일)	정답 y
1	1 ~ 8일	9일
2	2 ~ 9일	10일
3	3 ~ 10일	11일

윈도우 = 지난 W개 값

지난 8일(1~8일) 기온 → 9일째 기온
다음 문제는 한 칸 밀어서
2~9일 → 10일째

```
W = 8
X = [t[i:i+W] for i in range(N-W)]
y = [t[i+W]   for i in range(N-W)]
```

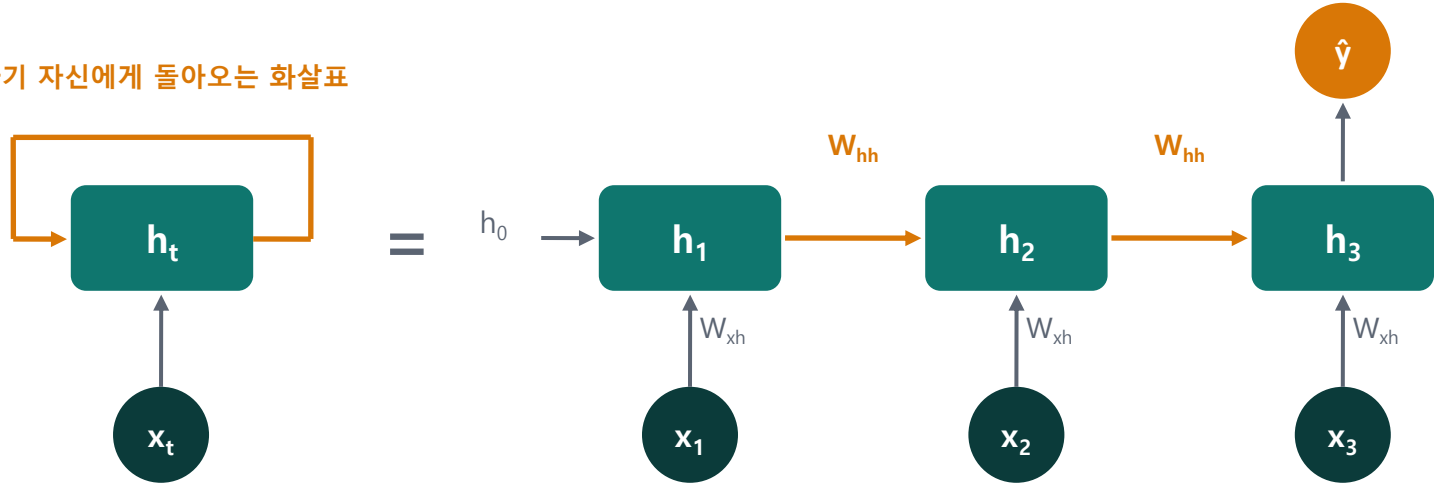
왜 RNN일까: MLP·CNN과 비교

	MLP	CNN	RNN
입력 방식	W개를 한 번에 펼쳐서	가까운 이웃끼리 묶어서	**앞에서부터 하나씩**
순서 처리	자리마다 다른 w (순서 개념 없음)	짧은 범위의 패턴	**이전 기억 h를 다음 시점에 전달**
입력 길이	고정	고정 (보통)	**자유 (문장 길이 달라도 됨)**
잘하는 것	표 형태 데이터	이미지	시계열, 문장, 음성

RNN = Recurrent(반복되는) Neural Network
같은 계산 셀을 시점마다 반복해서 쓰고, 셀의 결과(기억)를 다음 시점으로 넘김

RNN 구조: 접힌 그림과 펼친 그림

자기 자신에게 돌아오는 화살표



시간 순서로 펼치기 →

읽는 법

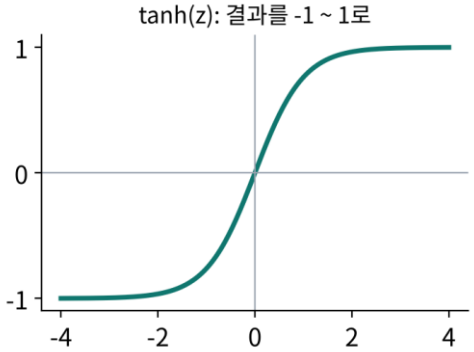
x_1부터 하나씩 넣음. 셀은 지금 입력 x_t와 이전 기억 h_{t-1}을 받아 새 기억 h_t를 만듦. 마지막 h_3로 예측값 $\hat{y}$ 계산.

RNN 한 칸의 계산

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

기호	의미	비유
x_t	지금 들어온 값 (오늘 기온)	지금 읽는 단어
h_{t-1}	직전까지의 기억 (hidden state)	앞 내용 요약
W_{xh}	입력을 기억으로 바꾸는 w	새 정보 반영 정도
W_{hh}	이전 기억을 넘기는 w	옛 기억 유지 정도
h_t	새 기억 → 다음 시점으로 전달	업데이트된 요약

예측값: $\hat{y} = W_{hy} \cdot h_T + b$ (마지막 기억으로 계산, 코드에서는 nn.Linear)



왜 tanh?

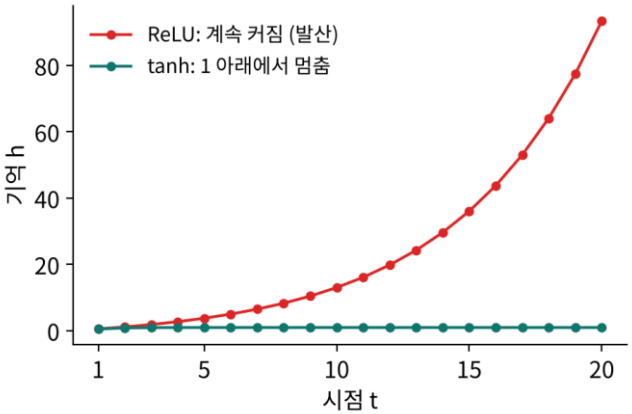
결과를 −1 ~ 1로 눌러 줌. ReLU를 쓰면 발산 → 다음 장에서 숫자로 확인.

왜 tanh일까: ReLU를 쓰면 발산

Vanilla RNN: 새 상태 h_t = f(이전 상태 h_{t-1}, 지금 입력 x_t) → 같은 W_{hh}가 매 시점 다시 곱해지는 구조

예제: W_{xh} = 0.5, W_{hh} = 1.2, b = 0, 입력 x = 1을 20번 넣기

시점	ReLU 사용	tanh 사용
1	0.50	0.462
2	1.10	0.784
5	3.72	0.922
10	**12.98**	0.923
20	**93.34**	**0.923**



이유

ReLU는 양수를 그대로 통과시켜 상한이 없음 → W_{hh} > 1이면 매 시점 1.2배씩 커져 발산. tanh는 −1 ~ 1에 묶여 반복해도 안정. (CNN은 층마다 다른 w를 한 번씩만 지나서 ReLU도 괜찮음)

대신 생기는 문제

tanh 미분은 최대 1 → 여러 번 곱하면 작아짐 → 장기 의존성 문제 (2일차 LSTM)

예제: RNN 손으로 계산하기

W_{xh} = 0.5, W_{hh} = 0.8, b = 0.1, h_0 = 0, 입력 x = [1.0, 0.5, −1.0]

t	x	$z = 0.5 \cdot x + 0.8 \cdot h_{t-1} + 0.1$	$h_t = \tanh(z)$
1	1.0	0.5×1.0 + 0.8×0 + 0.1 = 0.600	**0.537**
2	0.5	0.5×0.5 + 0.8×0.537 + 0.1 = 0.780	**0.652**
3	−1.0	0.5×(−1.0) + 0.8×0.652 + 0.1 = 0.122	**0.121**

```
rnn = nn.RNN(input_size=1, hidden_size=1, batch_first=True)
# w를 0.5, 0.8, 0.1로 직접 넣으면
out, h_n = rnn(torch.tensor([1.0, 0.5, -1.0]).reshape(1, 3, 1))
# out → [0.537, 0.652, 0.121]  손계산과 같음 (실습 1에서 확인)
```

포인트

모든 시점에서 같은 0.5, 0.8, 0.1을 씀. 앞 기억 0.652가 큰 양수라 $x_3 = -1$이 들어와도 h_3는 양수로 남음.

예제 2: 번역으로 보는 RNN (문제 설정)

영어 (입력)	원-핫	한국어 (출력)	원-핫
baseball	[1, 0, 0]	**야구**	[1, 0, 0]
bat	[0, 1, 0]	**배트**	[0, 1, 0]
ball	[0, 0, 1]	공	[0, 0, 1]

문제

입력 [baseball, bat] → 출력 [야구, 배트]
시점마다 단어 1개를 넣고 번역 1개를 받음
(many-to-many, 매 시점 출력)

크기: 입력 3 (단어 3개) · hidden 2 · 출력 3, h_0 = [0, 0], b는 0으로 생략

W_{xh} (2×3) 입력 → 기억

$$\begin{bmatrix} 0.5 & 0.1 & 0.2 \\ -0.3 & 0.4 & 0.1 \end{bmatrix}$$

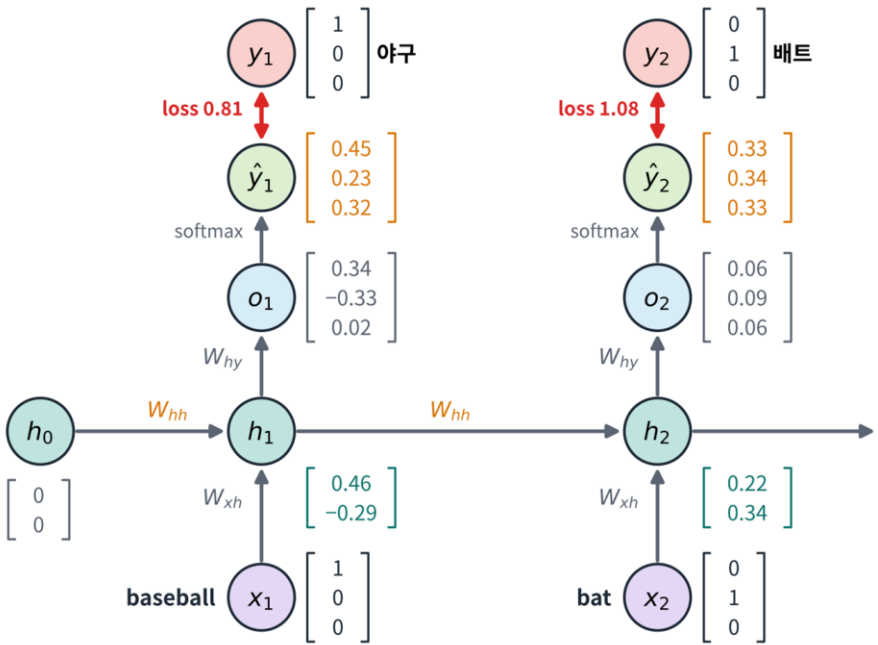
W_{hh} (2×2) 기억 → 기억

$$\begin{bmatrix} 0.2 & -0.1 \\ 0.1 & 0.3 \end{bmatrix}$$

W_{hy} (3×2) 기억 → 출력

$$\begin{bmatrix} 0.6 & -0.2 \\ -0.4 & 0.5 \\ 0.1 & 0.1 \end{bmatrix}$$

예제 2: 순전파 한눈에 보기



단계	식	t = 1 (baseball)
① 기억	$h = \tanh(W_{xh}x + W_{hh}h_{-1})$	[0.46, −0.29]
② 점수	$o = W_{hy}\,h$	[0.34, −0.33, 0.02]
③ 확률	$\hat{y} = \text{softmax}(o)$	[0.45, 0.23, 0.32]
④ 손실	−log(정답 칸 확률)	**−log 0.45 = 0.81**

결과

t=1: 야구 0.45로 1등 (맞았지만 확신 낮음)
t=2: 배트 0.34로 겨우 1등 (거의 반반)
전체 loss = 0.81 + 1.08 = 1.89

예제 2: 계산 따라가기 (t = 1, t = 2)

계산	t = 1 (x_1 = baseball)	t = 2 (x_2 = bat)
$W_{xh} \cdot x$	[0.5, −0.3] ← 1열 선택	[0.1, 0.4] ← 2열 선택
$W_{hh} \cdot h_{t-1}$	[0, 0] ($h_0 = 0$)	[0.2×0.46 − 0.1×(−0.29), 0.1×0.46 + 0.3×(−0.29)] = [0.12, −0.04]
z = 합	[0.5, −0.3]	[0.22, 0.36]
h = tanh(z)	**[0.46, −0.29]**	**[0.22, 0.34]**
$o = W_{hy} \cdot h$	[0.34, −0.33, 0.02]	[0.06, 0.09, 0.06]
$\hat{y}$ = softmax(o)	**[0.45, 0.23, 0.32]**	**[0.33, 0.34, 0.33]**
loss	−log 0.45 = 0.81	−log 0.34 = 1.08

원-핫 × 행렬 = 열 하나 고르기

x_1 = [1, 0, 0]이면 W_{xh}의 1열만 남음. 그래서 W_{xh}의 각 열이 곧 '그 단어의 벡터' 역할을 함 (나중에 배울 임베딩과 같은 아이디어).

예제 2: 역전파로 W 고치기

① 순전파 → ② loss → ③ 역전파 → ④ 업데이트

① 순전파: h, o, ŷ 계산

② loss: 1.89

③ 역전파: 세 W의 기울기

④ 업데이트: W ← W − lr × 기울기

W	미분이 오는 길	이 예제의 기울기 (lr 0.5)
W_{hy}	각 시점 출력에서 바로 ($o = W_{hy}h$)	3×2 행렬
W_{xh}	출력 → h → 입력 쪽	3열(ball)은 0: 안 쓴 단어는 안 바뀜
W_{hh}	**출력 → h_2 → h_1 (시간을 거슬러: BPTT)**	[[0.218, −0.138], [−0.148, 0.093]]

업데이트 예: W_{hh}의 왼쪽 위 칸

0.2 − 0.5 × 0.218 = 0.091
기울기가 양수 → 값을 줄이면 loss가 줄어듦

1번 업데이트 후

전체 loss 1.89 → 1.51
야구 확률 0.45 → 0.51, 배트 0.34 → 0.44

예제 2: 학습하면 어떻게 바뀌나

업데이트 횟수	전체 loss	t=1 ŷ (야구, 배트, 공)	t=2 ŷ (야구, 배트, 공)
0 (처음)	1.89	[0.45, 0.23, 0.32]	[0.33, 0.34, 0.33]
1	1.51	[0.51, 0.20, 0.30]	[0.24, 0.44, 0.32]
10	0.20	[0.90, 0.01, 0.09]	[0.01, 0.91, 0.08]
100	**0.02**	**[0.99, 0.00, 0.01]**	**[0.00, 0.99, 0.01]**

학습 = W를 찾는 것

처음 W는 아무 숫자(랜덤). 순전파 → loss → 역전파 → 업데이트를 반복하면 W가 바뀌면서 정답 칸의 확률이 1에 가까워짐. CNN에서 커널을 찾던 것과 같은 과정.

기억(W_{hh})이 필요한 이유: bat → 배트? 박쥐?

문장	t=1	t=2
baseball bat	야구	**배트**
cave bat	동굴	**박쥐**

두 번째에 들어오는 단어는 둘 다 bat

300번 학습 후	야구	배트	동굴	박쥐
W_{hh} 사용	1.00	**1.00**	1.00	**1.00**
W_{hh} = 0	1.00	**0.50**	1.00	**0.50**

정답 칸 확률

해석

W_{hh} = 0이면 bat만 보고 번역해야 해서 배트·박쥐를 반반(0.50)으로 찍을 수밖에 없음. W_{hh}가 있으면 h_1에 담긴 앞 단어(baseball / cave)의 기억이 h_2로 넘어와 구별함 → 이것이 Recurrent(순환)의 역할.

가중치 공유와 파라미터 수

	CNN	RNN
같은 w를 재사용하는 곳	이미지의 모든 위치 (커널)	**모든 시점**
입력이 커지면 w 개수는?	그대로	**그대로 (길이 100이어도 같음)**

파라미터 수 = H × I + H × H + H + H (I = 입력 크기, H = hidden 크기)

이름 (PyTorch)	모양	nn.RNN(1, 8)일 때
`weight_ih_l0` (W_xh)	(H, I)	8 × 1 = 8
`weight_hh_l0` (W_hh)	(H, H)	8 × 8 = 64
`bias_ih_l0, bias_hh_l0`	(H,) 2개	8 + 8 = 16
합계		**88**

nn.RNN의 입력·출력 모양

	모양 (batch_first=True)	의미
입력 x	(배치, 길이, 입력 크기)	윈도우 64개 × 30일 × 하루 값 1개
출력 out	(배치, 길이, H)	모든 시점의 h (h_1 ~ h_T)
h_n	(층 수, 배치, H)	마지막 시점의 h (h_T)

```
rnn = nn.RNN(input_size=1, hidden_size=8, batch_first=True)
x = torch.randn(4, 5, 1)        # 배치 4, 길이 5, 값 1개
out, h_n = rnn(x)
out.shape    # (4, 5, 8)
h_n.shape    # (1, 4, 8)
out[:, -1, :]  ==  h_n[0]        # 마지막 시점은 같은 값
```

다음 값 예측에서는

마지막 기억만 필요 → out[:, -1, :]를 nn.Linear(H, 1)에 넣어 숫자 1개 출력.

입출력 유형: 몇 개 넣고 몇 개 받나

one-to-many

1개 → 여러 개

사진 → 설명 문장

many-to-one

여러 개 → 1개

30일 기온 → 내일 기온
리뷰 문장 → 긍정/부정

many-to-many

여러 개 → 여러 개 (입력 다 읽고)

번역: 한국어 문장 → 영어 문장

many-to-many

매 시점마다 출력

문장의 단어마다 품사 표시
매일 다음 날 기온

```
# many-to-one
out, _ = self.rnn(x)
return self.fc(out[:, -1, :])    # 마지막 시점만

# many-to-many (매 시점)
return self.fc(out)              # 모든 시점
```

구분 기준

out에서 마지막 시점만 쓰나, 모든 시점을 쓰나의 차이.

1일차 문제

문제 1. 윈도우 만들기

데이터 [3, 5, 4, 6, 7, 5], 윈도우 3
(1) 만들어지는 (X, y) 쌍은 몇 개?
(2) 모든 쌍을 적기

문제 2. 손계산

W_xh = 1, W_hh = 0.5, b = 0, h_0 = 0
입력 x = [1, −1]
h_1, h_2를 구하기 (tanh(1) = 0.762, tanh(−0.619) = −0.551)

문제 3. 모양과 파라미터 수

nn.RNN(input_size=3, hidden_size=16, batch_first=True)
입력 x 모양 (32, 10, 3)
(1) out 모양 (2) h_n 모양 (3) 파라미터 수

문제 4. 입출력 유형

(1) 지난 7일 주가 → 내일 주가
(2) 영화 리뷰 → 별점
(3) 영어 문장 → 한국어 문장
(4) 사진 한 장 → 설명 문장

1일차 정답

문제 1

(1) 6 − 3 = 3개
(2) [3, 5, 4] → 6
[5, 4, 6] → 7
[4, 6, 7] → 5

문제 2

$h_1 = \tanh(1\times1 + 0.5\times0) = \tanh(1) = 0.762$
$z_2 = 1\times(-1) + 0.5\times0.762 = -0.619$
$h_2 = \tanh(-0.619) = -0.551$

문제 3

(1) out (32, 10, 16)
(2) h_n (1, 32, 16)
(3) 16×3 + 16×16 + 16 + 16 = 336

문제 4

(1) many-to-one
(2) many-to-one
(3) many-to-many (입력 다 읽고 출력)
(4) one-to-many

1

1일차 실습:
순전파·역전파, 사인파

실습1_순전파_역전파.ipynb · 실습2_사인파_RNN.ipynb

실습 1 순전파 · 역전파 (번역 예제)

순서	내용	시간	확인할 숫자
1	순전파: 슬라이드 예제 그대로	5분	loss 1.89
2	역전파: 세 W의 기울기, 1번 업데이트	10분	1.89 → 1.51
3	학습 반복 100번 + 그래프	5분	0.02, 확률 0.99
4	nn.RNN + SGD로 같은 학습	5분	같은 loss 나오는지
5	기억이 필요한 경우 (배트·박쥐)	5분	W_{hh} = 0이면 0.50
6	문제 7개: 코드 고쳐 보기	과제	학습률, 단어 추가, hidden 4 …

```
loss, _ = forward(W_xh, W_hh, W_hy, xs, targets)    # 순전파
loss.backward()                                        # 역전파 (BPTT)
with torch.no_grad():
    for W in (W_xh, W_hh, W_hy):
        W -= LR * W.grad                               # 업데이트
        W.grad = None
```

실습 2 사인파 RNN: 구성과 모델

순서	내용	시간
1	사인파 데이터와 윈도우 20 만들기	5분
2	nn.RNN shape, 파라미터 88개 확인	5분
3	RNN 모델 만들고 학습 (MSE)	10분
4	예측 결과 그래프	3분
5	문제 7개: 코드 고쳐 보기	과제

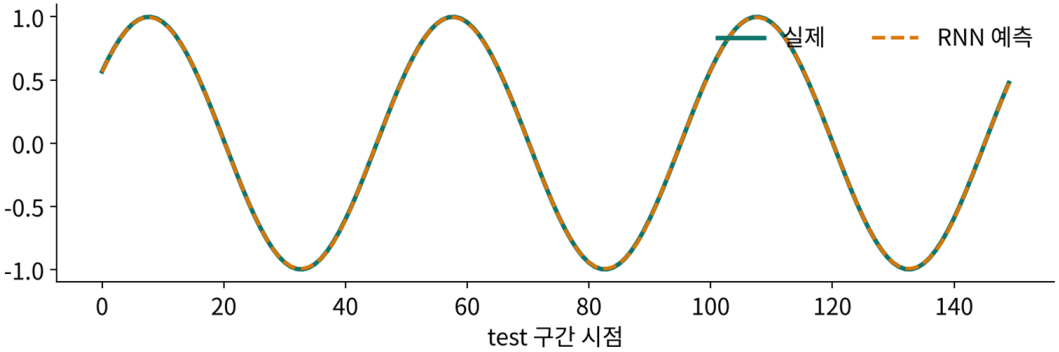
데이터

sin 곡선 1,000개 점 → 윈도우 20 → (980, 20, 1). 앞 80% 학습, 뒤 20% 테스트 (시간순).

```
class SineRNN(nn.Module):
  def __init__(self, hidden=16):
    super().__init__()
    self.rnn = nn.RNN(1, hidden,
                      batch_first=True)
    self.fc = nn.Linear(hidden, 1)
  def forward(self, x):      # (배치, 20, 1)
    out, h_n = self.rnn(x)   # (배치, 20, 16)
    return self.fc(out[:, -1, :])

loss_fn = nn.MSELoss()       # 회귀 → MSE
opt = torch.optim.Adam(model.parameters(),
                       lr=0.01)
```

실습 2 사인파 RNN: 결과



설정	test MSE	파라미터
기본 (윈도우 20, hidden 16)	**0.00001**	321
윈도우 5	0.00019	321
윈도우 50	0.00009	321
hidden 2	0.00105	13
hidden 64	0.000001	4,353
잡음 0.1 추가	0.0139	321

읽을거리

윈도우는 너무 짧아도(정보 부족), 너무 길어도(학습 어려움) 나빠짐 → 2일차 '기울기 소실'로 이어짐. 잡음은 예측할 수 없어서 MSE가 잡음 크기(0.01) 아래로 안 내려감.

문제 정답은 노트북 마지막 '정답 보기'에 있음

2

2일차: 시계열 예측과 RNN 학습

데이터 준비 · 평가 · 장기 의존성 · LSTM · GRU

시계열 예측 준비 4단계

① **시간순 분할**

과거로 학습, 미래로 평가
train → val → test

→

② **정규화**

train의 최솟값·최댓값으로
0 ~ 1로 맞춤

→

③ **윈도우**

지난 W일 → 다음 날
(입력, 정답) 쌍 만들기

→

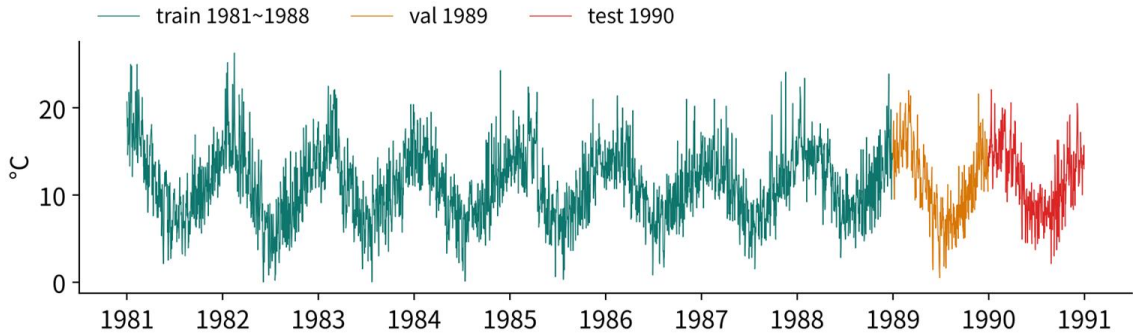
④ **배치는 섞기**

윈도우끼리는 섞어도 됨
shuffle=True

자주 하는 실수

· 분할 전에 전체 데이터로 정규화 → test 범위를 미리 봄 (데이터 누수)
· 윈도우를 만든 뒤 random_split으로 섞어 나눔 → 1990년 날짜가 train에 섞여 점수가 부풀려짐
· 분할은 시간순, 배치는 섞기: 둘을 헷갈리지 않기

시간순 분할과 데이터 누수



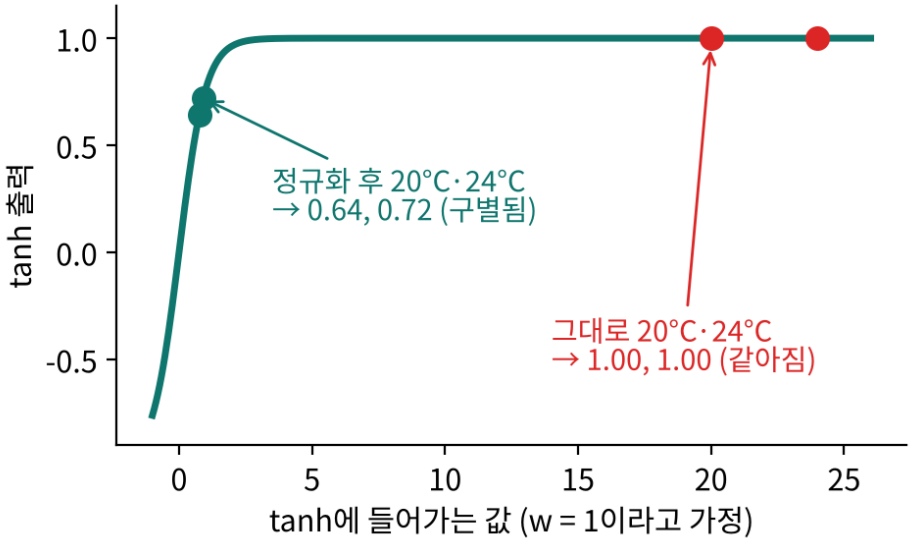
왜 섞으면 안 될까

1990년 3월 5일이 train에 있고 3월 6일이 test에 있으면, 모델은 바로 전날 값을 외워서 맞힘 → 실제보다 좋은 점수.

데이터 누수(leakage): 평가에 써야 할 정보가 학습·전처리에 새어 들어가는 것

누수 예시	올바른 방법
전체 데이터로 min·max 정규화	train으로만 min·max 계산, val·test에 같은 값 적용
윈도우를 섞은 뒤 train/test 분할	날짜 기준으로 먼저 자르고 윈도우 만들기

정규화는 왜 할까



① tanh 포화 막기

기온 20을 그대로 넣으면 tanh가 1에 붙어 버림. 20°C와 24°C가 둘 다 1.00이 되어 구별 못 하고, 미분도 0에 가까워 학습이 멈춤.

② 학습을 안정적으로

입력·정답이 0~1 근처면 작은 w 초깃값, 학습률 0.001~0.01과 잘 맞음. 값이 크면 첫 loss와 기울기가 커서 학습이 들쭉날쭉.

③ 단위 맞추기

기온(°C)·습도(%)·기압(1000 hPa 근처)을 같이 넣을 때 숫자가 큰 기압이 결과를 좌우하지 않게 같은 범위로 맞춤.

주의: 최솟값·최댓값은 train 데이터로만 구함 (전체로 구하면 데이터 누수). 예측값은 °C로 되돌려서 평가.

정규화: 0 ~ 1로 맞추고 다시 되돌리기

정규화 $x' = (x - \min) / (\max - \min)$　　되돌리기 $x = x' \times (\max - \min) + \min$

예제 (실습 데이터의 train 기준: min 0.0, max 26.3)	계산	결과
20.0 °C를 정규화	(20.0 − 0) / 26.3	**0.760**
모델 출력 0.5를 °C로	0.5 × 26.3 + 0	**13.15 °C**
test에 27.5 °C가 있으면?	(27.5 − 0) / 26.3	1.046 (1보다 커도 괜찮음)

되돌려서 평가

모델은 0~1 값을 예측함. 되돌리기 식으로 °C로 바꾼 뒤 MAE를 계산해야 '평균 몇 도 틀림'으로 읽을 수 있음.

기준선(baseline)과 평가 지표

지표	계산	단위	특징
MSE	(예측 − 실제)²의 평균	°C²	학습 손실로 사용, 큰 오차에 민감
MAE	|예측 − 실제|의 평균	°C	**평가에 사용, '평균 몇 도 틀림'으로 읽기 쉬움**

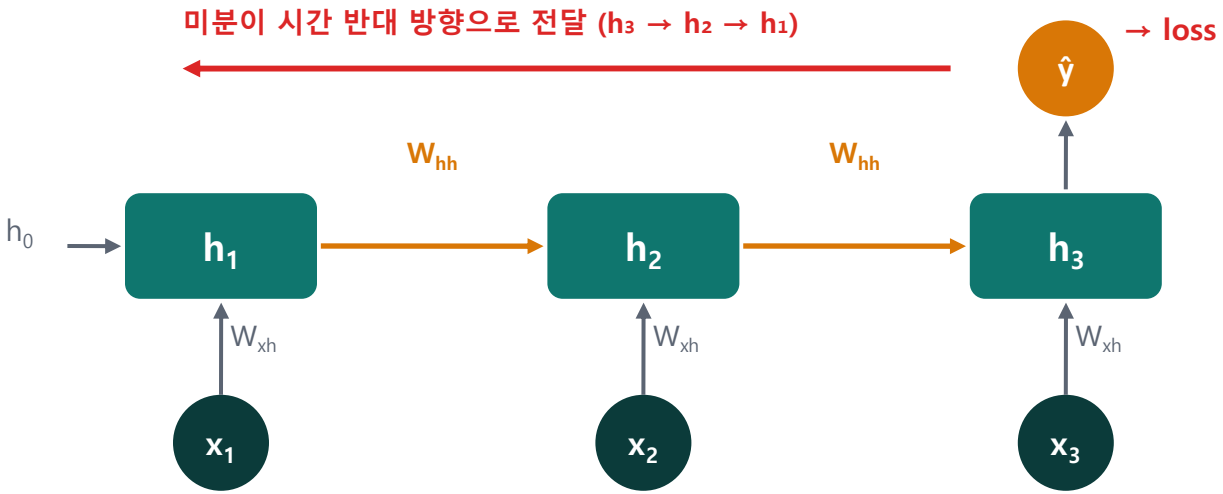
기준선: 내일 = 오늘

아무것도 학습하지 않은 가장 단순한 예측. 실습 데이터 test MAE = 2.02 °C
→ RNN이 2.02 °C보다 작아야 의미가 있음.

예제: 기준선 MAE

기온 [10, 12, 11, 15]
예측(전날 값): 10, 12, 11 / 실제: 12, 11, 15
오차 2, 1, 4 → MAE = 7 / 3 = 2.33 °C

RNN 학습: 펼친 그림 위의 역전파 (BPTT)



$\partial L/\partial h_1 = \partial L/\partial h_3 \times \partial h_3/\partial h_2 \times \partial h_2/\partial h_1$

한 칸마다 곱해지는 값
= $W_{hh} \times \tanh'(z)$

체인룰 복습과 같은 구조

역전파 시간에 $\partial L/\partial w_1$ = 여러 미분의 곱이었던 것처럼, RNN은 시점을 거슬러 갈 때마다 같은 모양의 미분이 계속 곱해짐. 같은 W를 쓰므로 미분도 시점마다 모두 더해서 W를 한 번 업데이트.

장기 의존성 문제 (기울기 소실)

한 칸마다 곱해지는 값	10칸 뒤	20칸 뒤	결과
0.5	0.001	0.000001	**거의 0 → 먼 과거는 학습 안 됨**
0.8	0.107	0.0115	20일 전 영향은 약 1%
1.2	6.2	38.3	**반대로 폭발 (기울기 폭주)**

증상

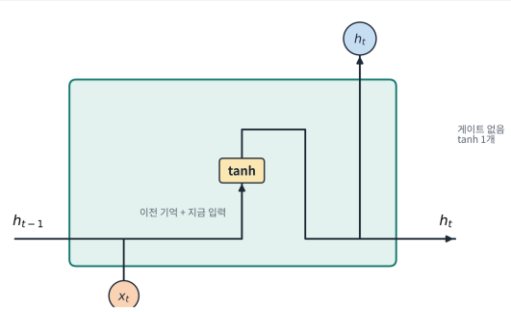
윈도우를 길게 해도 앞부분 정보를 못 씀. 1일차 사인파에서 윈도우 50이 20보다 나빴던 이유 중 하나.

해결 방법

· 기울기 폭주: 기울기 크기를 잘라 줌 (gradient clipping)
· 기울기 소실: 구조를 바꿈 → LSTM, GRU

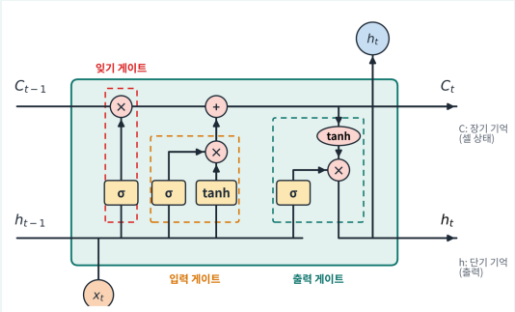
그래서 등장: LSTM과 GRU

RNN (Vanilla)



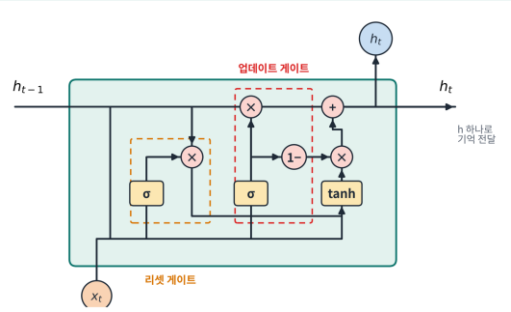
tanh 1개, 게이트 없음
먼 과거 기억이 빨리 사라짐

LSTM (1997)



게이트 3개 + 셀 상태 C
장기 기억 통로를 따로 둠

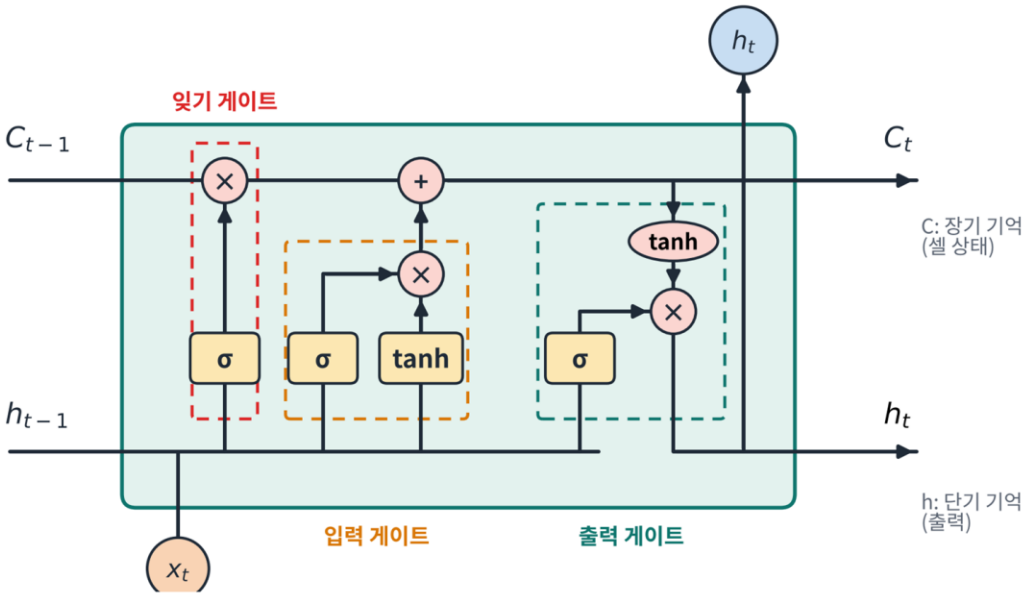
GRU (2014)



게이트 2개, 상태는 h 하나
LSTM을 간단하게

공통 아이디어: σ(0 ~ 1) 게이트로 '얼마나 지우고, 얼마나 넣을지'를 학습 → 기억을 곱셈이 아닌 덧셈으로 이어 기울기가 덜 사라짐

LSTM ① 구조: 장기 기억 C와 게이트 3개



게이트	하는 일
잊기 (forget)	이전 기억 C에서 무엇을 버릴지
입력 (input)	새 정보 중 무엇을 C에 저장할지
출력 (output)	C 중 무엇을 h로 내보낼지

기억이 2개

C: 장기 기억 (셀 상태)
h: 단기 기억 (출력, 다음 층·예측에 사용)

σ = 문 열림 정도

0이면 완전히 닫힘(막음), 1이면 완전히 열림(통과). 학습으로 언제 열고 닫을지 정함.

LSTM ② 계산 순서와 숫자 예제

단계	식	의미
① 잊기	$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b)$	이전 기억 유지 비율
② 입력	$i = \sigma(\ldots)$, $\tilde{C} = \tanh(\ldots)$	새 정보 비율, 새 정보 후보
③ 업데이트	**$C_t = f \times C_{t-1} + i \times \tilde{C}$**	**장기 기억 갱신**
④ 출력	$o = \sigma(\ldots)$, $h_t = o \times \tanh(C_t)$	내보낼 단기 기억

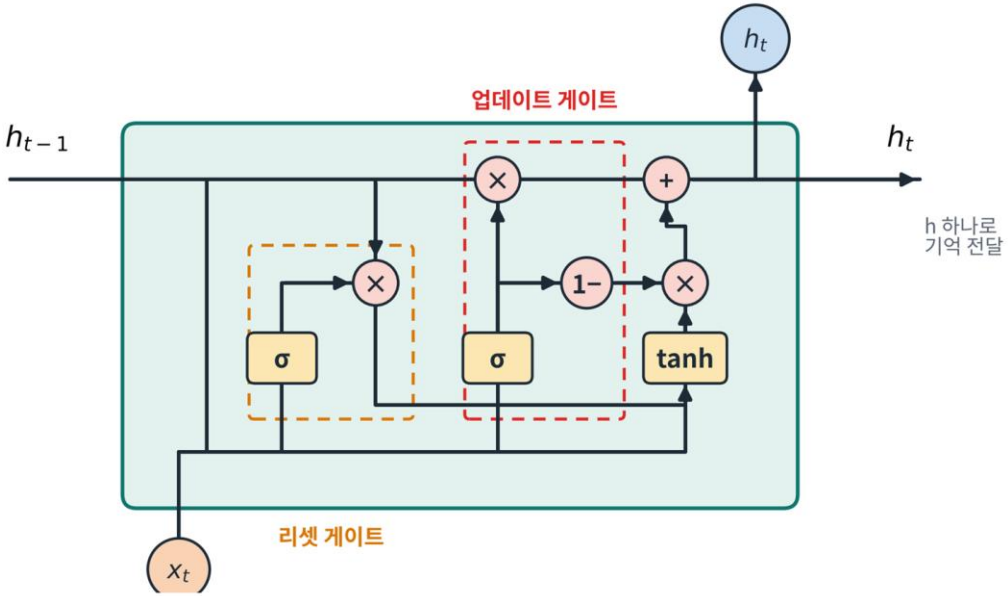
예제 A: 잘 기억하기

$C_{t-1} = 2.0$, $f = 0.9$, $i = 0.5$, $\tilde{C} = 0.8$, $o = 0.5$
$C_t = 0.9 \times 2.0 + 0.5 \times 0.8 = 2.2$
$h_t = 0.5 \times \tanh(2.2) = 0.5 \times 0.976 = 0.488$

예제 B: 잊어버리기

같은 값에서 $f = 0.1$로만 바꾸면
$C_t = 0.1 \times 2.0 + 0.5 \times 0.8 = 0.6$
→ 이전 기억 2.0이 거의 지워짐

GRU: 게이트 2개로 간단하게



게이트	하는 일
리셋 (reset) r	새 후보 만들 때 과거를 얼마나 무시할지
업데이트 (update) z	과거와 새 정보의 비율 결정

$$h_t = z \times h_{t-1} + (1 - z) \times \tilde{h_t}$$

예제

h_{t-1} = 1.0, 새 후보 $\tilde{h}$ = 0.2, z = 0.8
h_t = 0.8×1.0 + 0.2×0.2 = 0.84
→ 옛 기억을 80% 유지

RNN vs LSTM vs GRU 비교

	RNN	LSTM	GRU
게이트	없음	3개 (잊기·입력·출력)	2개 (리셋·업데이트)
전달하는 상태	h	h + C (장기 기억)	h
파라미터 수 (입력 1, hidden 32)	1,120 (×1)	4,480 (×4)	3,360 (×3)
먼 과거 기억	**약함**	**강함**	**강함**
속도·크기	가장 가벼움	가장 무거움	중간

```
nn.RNN(1, 32, batch_first=True)     # out, h_n = rnn(x)
nn.GRU(1, 32, batch_first=True)     # out, h_n = gru(x)          ← RNN과 같음
nn.LSTM(1, 32, batch_first=True)    # out, (h_n, c_n) = lstm(x)  ← C도 돌려줌
```

many-to-one에서는 세 모델 모두 out[:, -1, :]를 쓰면 되므로 코드가 거의 같음

2일차 문제

문제 1. 누수 찾기

다음 중 데이터 누수가 있는 것은?
(가) train으로 min·max를 구해 test도 변환
(나) 전체 기간으로 min·max를 구해 정규화
(다) 윈도우 만든 뒤 random_split으로 분할
(라) 학습 DataLoader에 shuffle=True

문제 2. 장기 의존성

한 칸마다 미분이 0.9씩 곱해진다면 30칸 뒤에는
처음의 몇 %? ($0.9^{30} \approx 0.042$)
이 문제를 무엇이라 하고, 어떤 모델로 줄일까?

문제 3. LSTM 셀 상태

$C_{t-1} = 1.5$, 잊기 $f = 0.6$,
입력 $i = 0.5$, 새 정보 $\tilde{C} = 1.0$일 때 C_t는?
f가 0이면 어떤 의미일까?

문제 4. 파라미터 수

입력 크기 1, hidden 16일 때
nn.RNN, nn.GRU, nn.LSTM의 파라미터 수는?
(RNN = 16×1 + 16×16 + 16 + 16)

2일차 정답

문제 1

(나), (다)
(나) test 기간의 범위를 미리 봄
(다) 이웃한 날이 train·test에 섞임
(가)는 올바른 방법, (라)는 분할이 아니라 배치 순서라 괜찮음

문제 2

약 4.2% → 30일 전 정보는 거의 반영 안 됨
장기 의존성 문제 (기울기 소실)
LSTM 또는 GRU

문제 3

$C_t = 0.6 \times 1.5 + 0.5 \times 1.0 = 1.4$
f = 0이면 이전 기억을 완전히 지움

문제 4

RNN 16 + 256 + 32 = 304
GRU 304 × 3 = 912
LSTM 304 × 4 = 1,216

2

2일차 실습:
LSTM·GRU로 바꿔 보기

실습3_사인파_LSTM_GRU.ipynb · 실습4_기온예측.ipynb

실습 3 사인파: RNN → LSTM·GRU 바꿔 보기

```
for Cell in [nn.RNN, nn.LSTM, nn.GRU]:
    model = SeqModel(Cell)       # 이름만 바꿈
    ...

# LSTM만 반환값이 다름
out, h_n = rnn(x)              # RNN, GRU
out, (h_n, c_n) = lstm(x)      # LSTM (c = 셀 상태)
```

모델	파라미터	윈도우 20	윈도우 100
RNN	321	0.000008	**0.000065**
LSTM	1,233 (×4)	0.000002	**0.000005**
GRU	929 (×3)	0.000006	**0.000012**

test MSE, 작을수록 좋음

읽을거리

윈도우 20에서는 셋 다 잘 맞힘. 윈도우 100으로 길어지면 RNN은 약 8배 나빠지지만 LSTM·GRU는 거의 그대로 → 순서가 길수록 게이트 구조가 유리. 이론 시간의 장기 의존성 문제를 직접 확인.

학습 시간은 환경마다 다름 (PyTorch는 LSTM 계산이 최적화돼 있어 오히려 빠를 때도 있음)

실습 4 기온 예측: 데이터와 설정

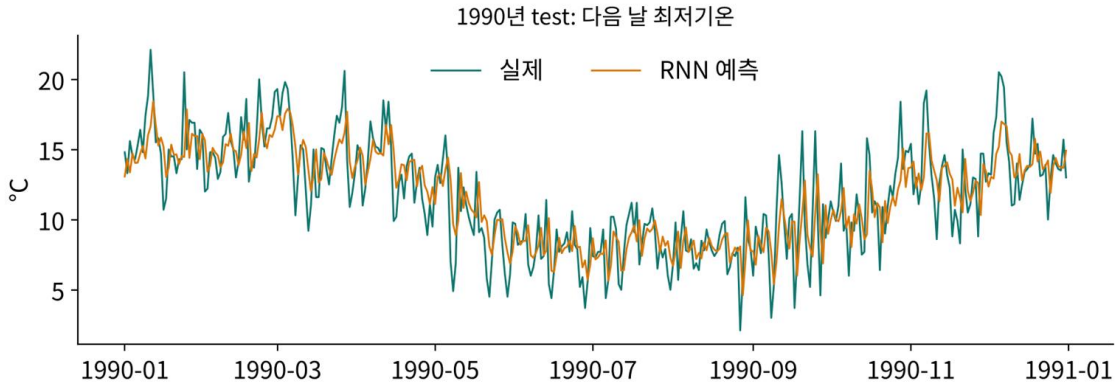
항목	설정
데이터	호주 멜버른 일별 최저기온 1981~1990 (3,650일)
분할	train 1981~1988 / val 1989 / test 1990 (시간순)
정규화	train 기준 min 0.0, max 26.3 → 0 ~ 1
윈도우	지난 30일 → 다음 날 (many-to-one)
모델	RNN / LSTM / GRU (hidden 32) → Linear(32 → 1)
학습	MSELoss, Adam lr 0.001, 배치 64 (shuffle), 30 에폭
평가	val MAE가 가장 좋을 때 저장 → test MAE (°C)

```
X_train, y_train = make_windows(0, i_val)
X_val, y_val = make_windows(i_val, i_test)
X_test, y_test = make_windows(i_test, len(s))
# X_train (2890, 30, 1)
# X_val   (365, 30, 1)
# X_test  (365, 30, 1)
```

확인 포인트

val·test 윈도우의 입력은 앞 해의 마지막 30일을 씀. 입력으로 과거를 보는 건 괜찮고, 맞힐 날만 겹치지 않으면 됨.

실습 4 기온 예측: 결과



기준선	RNN	LSTM	GRU
2.02	1.74	1.77	1.76

test MAE (°C), 작을수록 좋음

그래프

큰 흐름(계절)은 잘 따라가지만, 하루하루의 급변(전선, 비)은 늦게 따라감. 과거 기온만으로는 알 수 없는 정보이기 때문.

해석

세 모델 모두 기준선보다 약 0.25~0.3 °C 덜 틀림. 윈도우 30일처럼 짧으면 장기 기억이 필요 없어 LSTM·GRU의 장점이 드러나지 않음. 파라미터는 RNN 1,153 / GRU 3,393 / LSTM 4,513개.

정리

01	**시계열**	순서가 정보. 윈도우로 '지난 W개 → 다음 값' 문제를 만듦
02	**RNN**	$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$, 같은 w를 모든 시점에 재사용
03	**모양**	입력 (배치, 길이, 크기) → out 모든 시점, h_n 마지막 시점
04	**데이터 준비**	시간순 분할 → train 기준 정규화 → 윈도우, 배치만 섞기
05	**평가**	기준선(내일 = 오늘)과 MAE로 비교. 실습: 기준선 2.02 → RNN 1.74 ℃
06	**LSTM · GRU**	장기 의존성(기울기 소실) 해결: LSTM 게이트 3개 + 셀 상태, GRU 게이트 2개